Activitat 4: Github

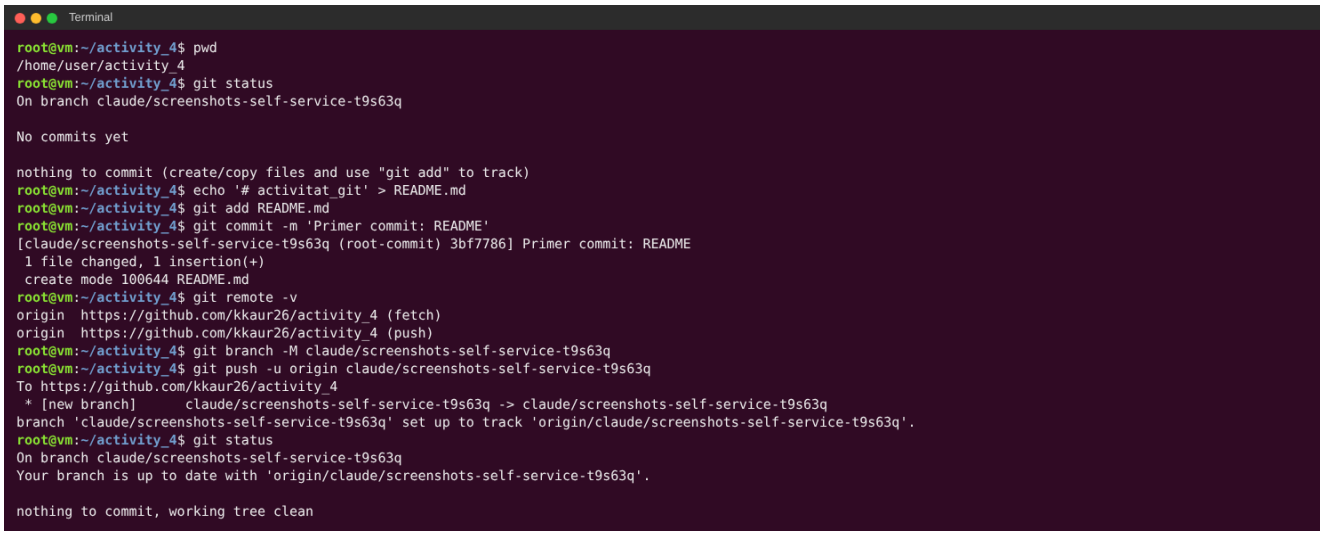
Repositori: https://github.com/kkaur26/activity_4 — branca de treball: claude/screenshots-self-service-t9s63q

Nota sobre l'entorn: totes les comandes s'han executat de veritat contra GitHub. Com que el repositori remot ja existia (kkaur26/activity_4, amb «origin» configurat), a l'apartat A1 es mostra «git remote -v» en lloc de «git remote add». Tant la part A com la B fan servir aquest mateix repositori i branca. El «company» se simula amb un altre clon en una altra carpeta (~/company i ~/company2) i amb un altre nom d'usuari de git.

Github A

1. Crear el projecte i pujar el projecte local

git add / commit, comprovació del remot, git branch -M i git push -u.



2. Afegir un fitxer, fer commit i git push origin



3. Crear una carpeta buida i clonar el projecte

La carpeta ~/company és buida (ls -A no mostra res) i s'hi fa git clone.

```
Terminal
root@vm:~$ mkdir company && cd company
root@vm:~/company$ ls -A
root@vm:~/company$ git clone -b claude/screenshots-self-service-t9s63q https://github.com/kkaur26/activity_4 activitat_git
Cloning into 'activitat_git'...
root@vm:~/company$ cd activitat_git
root@vm:~/company/activitat_git$ ls
README.md
fitxer_a2.txt
root@vm:~/company/activitat_git$ git log --oneline
5e9b551 Afegeix fitxer_a2.txt
3bf7786 Primer commit: README
root@vm:~/company/activitat_git$ git status
On branch claude/screenshots-self-service-t9s63q
Your branch is up to date with 'origin/claude/screenshots-self-service-t9s63q'.

nothing to commit, working tree clean
```

4. Canvis d'un altre company i git pull

Primer el «company» fa canvis des de l'altra carpeta i els publica:

```
Terminal
root@vm:~/company/activitat_git$ git config user.name 'Company (simulat)'
root@vm:~/company/activitat_git$ git config user.email company@example.com
root@vm:~/company/activitat_git$ echo 'Canvi fet pel company des d'un altre ordinador' >> README.md
root@vm:~/company/activitat_git$ git commit -am 'Company: actualitza README'
[claude/screenshots-self-service-t9s63q 4d7d7ac] Company: actualitza README
 1 file changed, 1 insertion(+)
root@vm:~/company/activitat_git$ git push origin claude/screenshots-self-service-t9s63q
To https://github.com/kkaur26/activity_4
   5e9b551..4d7d7ac  claude/screenshots-self-service-t9s63q -> claude/screenshots-self-service-t9s63q
```

Després, a la carpeta local original, obtenim els canvis amb git pull:

```
Terminal
root@vm:~/activity_4$ git status
On branch claude/screenshots-self-service-t9s63q
Your branch is up to date with 'origin/claude/screenshots-self-service-t9s63q'.

nothing to commit, working tree clean
root@vm:~/activity_4$ git pull origin claude/screenshots-self-service-t9s63q
From https://github.com/kkaur26/activity_4
 * branch            claude/screenshots-self-service-t9s63q -> FETCH_HEAD
   5e9b551..4d7d7ac  claude/screenshots-self-service-t9s63q -> origin/claude/screenshots-self-service-t9s63q
Updating 5e9b551..4d7d7ac
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
root@vm:~/activity_4$ cat README.md
# activitat_git
Canvi fet pel company des d'un altre ordinador
root@vm:~/activity_4$ git log --oneline
4d7d7ac Company: actualitza README
5e9b551 Afegeix fitxer_a2.txt
3bf7786 Primer commit: README
root@vm:~/activity_4$ git status
On branch claude/screenshots-self-service-t9s63q
Your branch is up to date with 'origin/claude/screenshots-self-service-t9s63q'.

nothing to commit, working tree clean
```

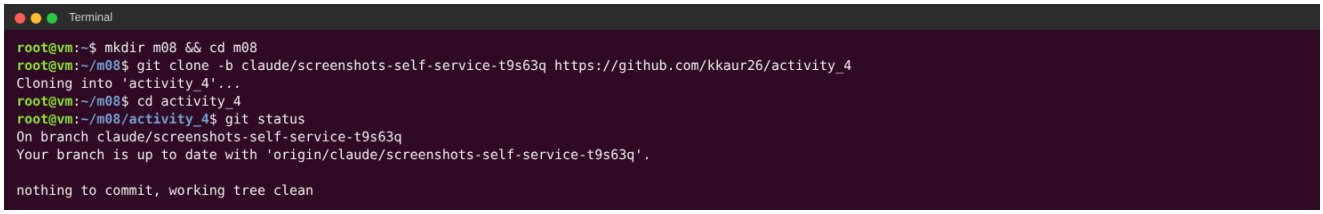
Github B

1. Crear un repositori al GitHub

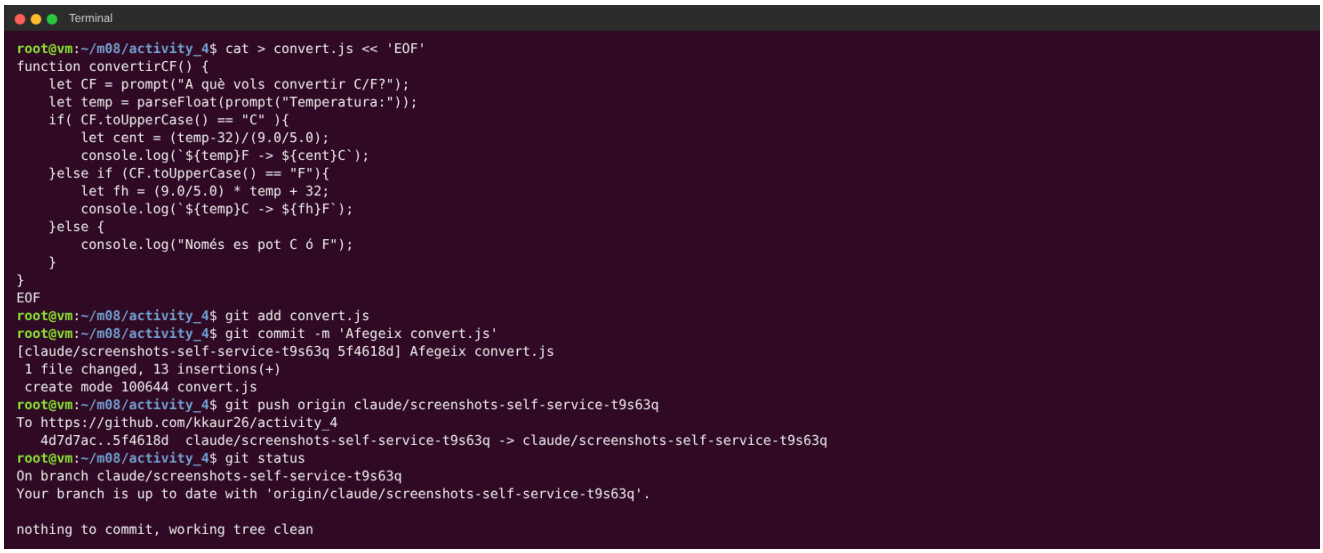
El repositori kkaur26/activity_4 existeix a GitHub; ho comprovem amb git ls-remote:

```
Terminal
root@vm:~/activity_4$ git ls-remote --heads origin
4d7d7ac58a31664229c64634ebc98518843780a0        refs/heads/claude/screenshots-self-service-t9s63q
```

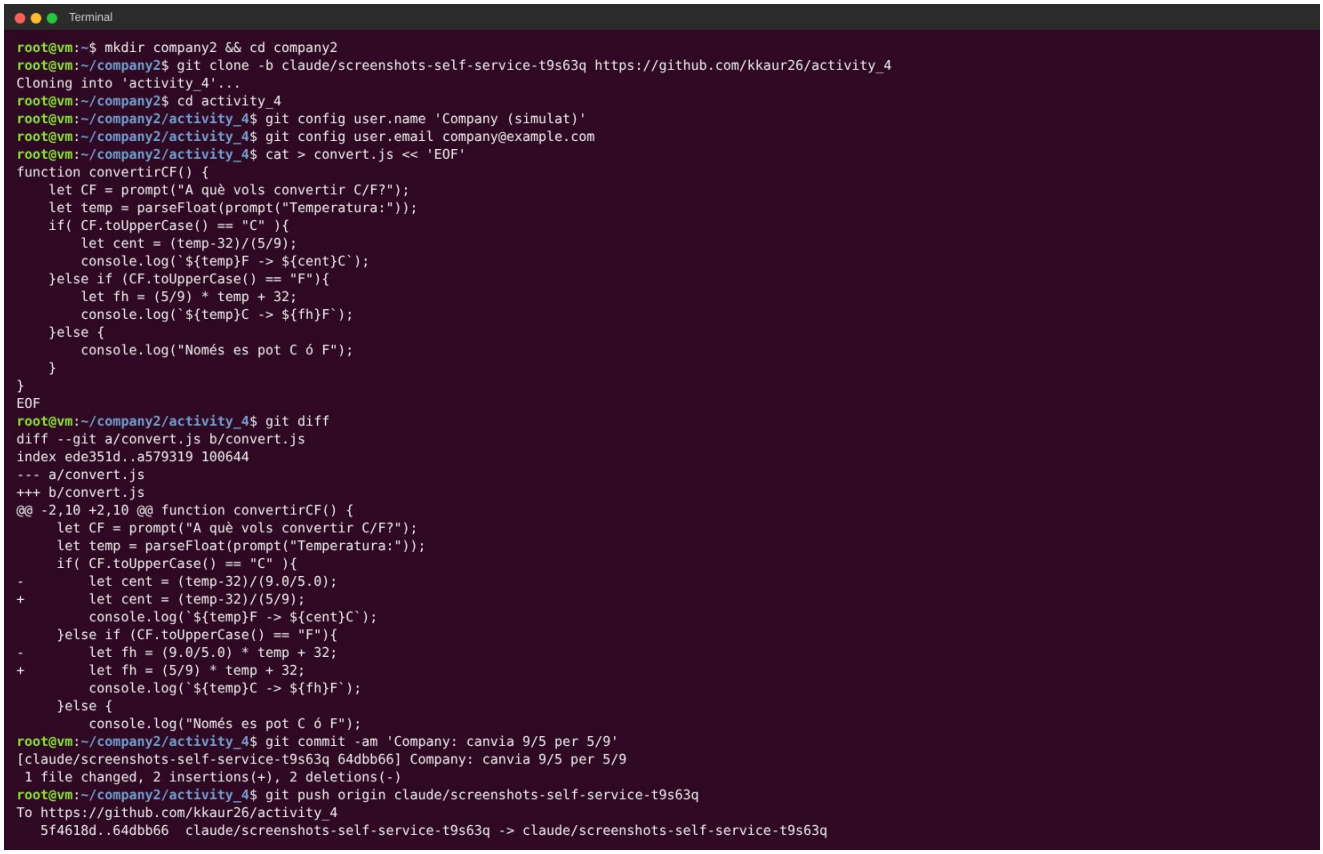
2. git clone a la carpeta local m08



3. Afegir convert.js i pujar-ho tot

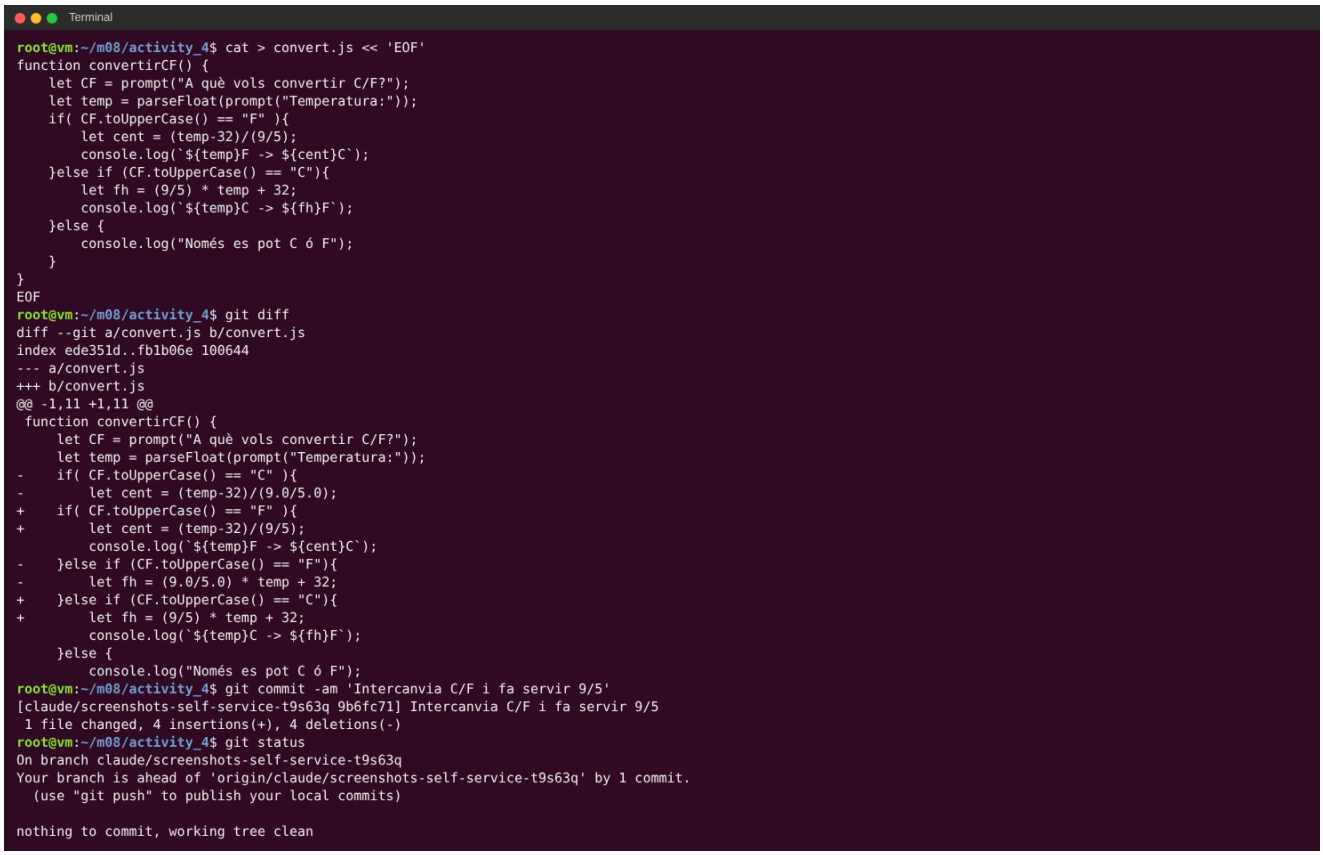


4. Un altre company clona el repositori i puja els seus canvis

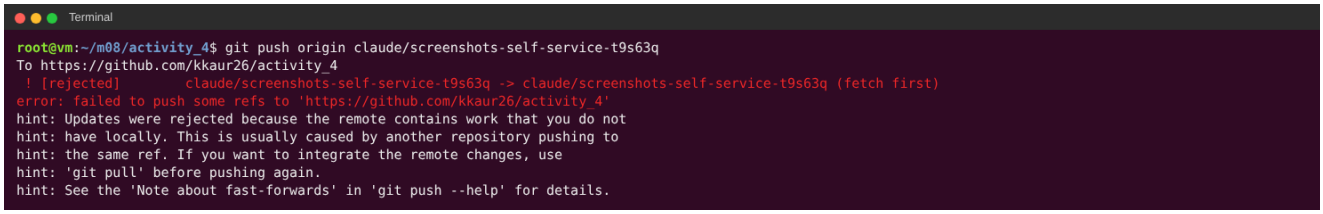


5. Ara tu fas els teus canvis

Fem els canvis a ~/m08 i en fem commit (encara sense push):



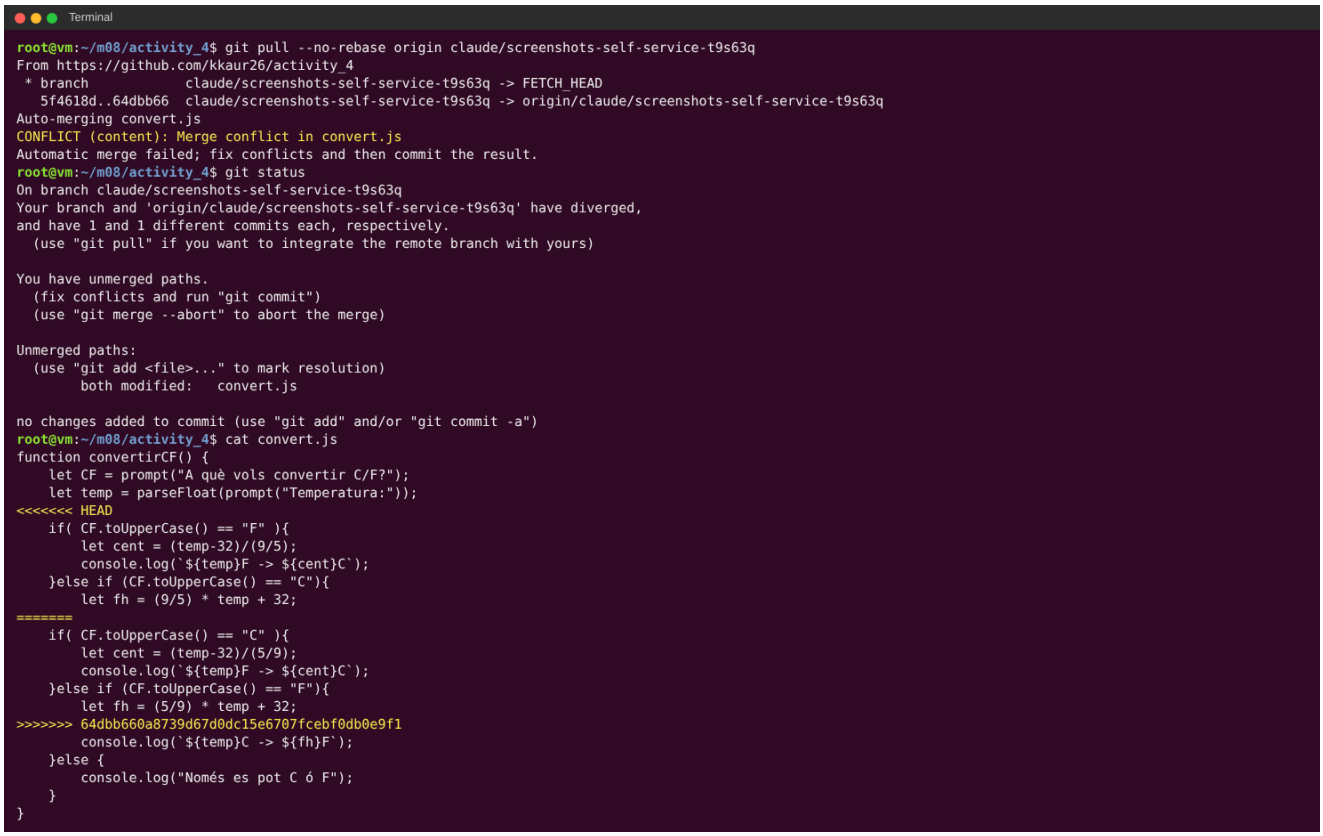
6. Intentar pujar els canvis. Et deixa? Per què?



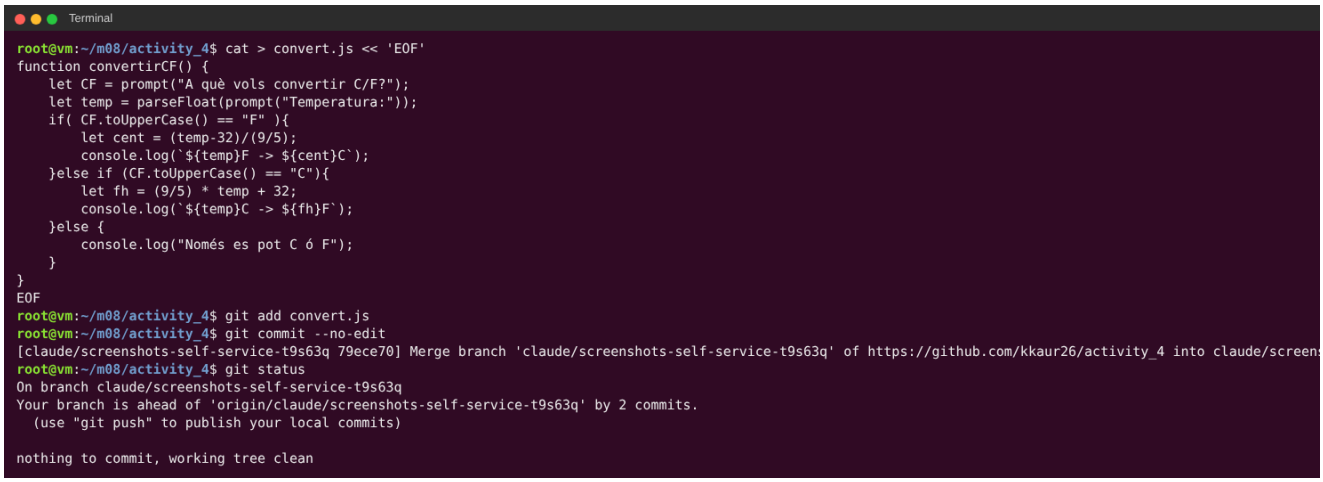
Resposta: No. GitHub rebutja el push («rejected … fetch first») perquè la branca remota conté un commit del company (64dbb66) que nosaltres no tenim en local. El nostre historial i el del remot han divergit, i un push normal només s'accepta si és un «fast-forward». Git no ens deixa sobreescriure la feina del company: primer hem d'integrar els seus canvis amb git pull.

7. Fer pull i revisar el codi: hi ha conflictes? Resoldre'ls

Sí, hi ha conflicte a convert.js, perquè tots dos hem modificat les mateixes línies:

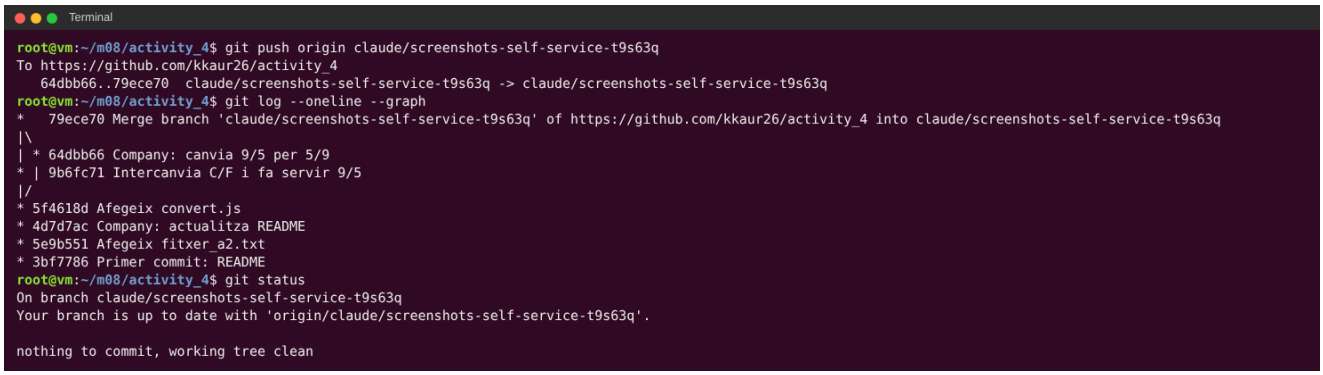


Resolem el conflicte: eliminem les marques <<<<<<< ======= >>>>>>> i deixem la versió correcta (si es demana F es passa a Fahrenheit→Celsius dividint per 9/5, i si es demana C es fa Celsius→Fahrenheit multiplicant per 9/5; la versió del company amb 5/9 dona resultats incorrectes). Després git add i git commit per tancar el merge:



8. Ara pots fer push? Fes-ho

Sí, ara el nostre historial ja conté el commit del company i el push és fast-forward:



9. És un incident o part del treball en grup habitual?

Resposta: És part normal del treball en grup. Quan diverses persones treballen sobre el mateix fitxer és habitual que el push sigui rebutjat o que aparegui un conflicte de merge. No és cap error de Git: és el mecanisme que evita que es perdi feina. Només esdevé un problema si es resol malament (per exemple, amb git push --force o acceptant una versió a cegues sense revisar-la).

10. Com minimitzar aquestes situacions en un grup molt gran?

Resposta: Faria servir un flux basat en branques (feature branches / GitHub Flow): 1) la branca main està protegida i ningú hi fa push directament; 2) cada tasca es fa en una branca pròpia i curta (feature/nom-tasca) creada des de main actualitzada; 3) es fan commits petits i freqüents, i es fa git pull/rebase de main sovint per detectar conflictes aviat; 4) quan la tasca està acabada s'obre un Pull Request, que passa proves automàtiques (CI) i revisió de codi d'un company; 5) s'integra a main quan està aprovat i s'esborra la branca. A més, ajuda repartir la feina perquè dues persones no toquin el mateix fitxer alhora, tenir un estil de codi comú (linters/formatadors) i comunicar-se (issues, tauler de tasques) per saber qui treballa en què.